

Universidade Federal do Cariri - Centro de Ciencias e Tecnologia

Logica para Ciencia da Computacao - 2026.1

Formalizacao em SAT: Problema 11 - Slitherlink

Modelagem proposicional, refinamento CEGAR e avaliacao experimental

Arthur Oliveira

Joao Caique

Gustavo Alexandre dos Santos

Professor: Luis Henrique Bustamante

Repositorio: github.com/santos-ag/slitherlink-solver

30 de agosto de 2026

Resumo. Este trabalho formaliza o Slitherlink em Forma Normal Conjuntiva (FNC). A codificacao usa uma variavel por aresta, restricoes de cardinalidade nas celulas e grau 0 ou 2 nos vertices. Como essas restricoes locais admitem varios ciclos desconectados, empregamos refinamento iterativo: modelos invalidos geram clausulas de bloqueio ate restar um unico laco. Seis instancias foram avaliadas com CaDiCaL e Kissat, incluindo casos SAT, UNSAT e um caso que exige multiplas rodadas de refinamento.

1. Introducao

Slitherlink e um puzzle de percurso sobre uma grade retangular de celulas. Algumas celulas possuem uma dica numerica. O objetivo e selecionar arestas da grade que formem um unico laco fechado, sem cruzamentos nem ramificacoes, de modo que cada dica seja igual ao numero de arestas selecionadas ao redor de sua celula.

O problema combina restricoes locais e uma propriedade global. As dicas e os graus dos vertices podem ser expressos por clausulas pequenas, mas a exigencia de um unico componente conexo depende de toda a solucao. Essa separacao torna o problema adequado ao estudo de SAT: a formula base captura a estrutura local e um processo de refinamento corrige modelos que ainda possuem subtours.

1.1 Regras consideradas

1. Uma celula com dica k possui exatamente k das quatro arestas ativas.
2. Cada vertice possui grau 0 ou 2. Assim, nao existem pontas soltas, cruzamentos ou ramificacoes.
3. Ao menos uma aresta deve estar ativa, eliminando a solucao vazia.
4. Todas as arestas ativas pertencem ao mesmo componente conexo.

1.2 Interesse computacional

O Slitherlink e NP-completo. Uma atribuicao candidata pode ser verificada em tempo polinomial: basta contar as arestas das celulas, calcular os graus e percorrer o grafo ativo. Entretanto, encontrar essa atribuicao pode exigir busca exponencial. Solvers CDCL exploram propagacao unitaria, aprendizado de clausulas e heurísticas de decisao para reduzir esse espaco.

O projeto utiliza dois solvers modernos, CaDiCaL e Kissat. A comparacao nao busca estabelecer superioridade geral, pois as instancias sao pequenas, mas observar diferencas de modelos iniciais, quantidade de cortes e custo de inicializacao.

Escopo. O Problema 11 (Slitherlink) foi selecionado da lista de problemas da disciplina. A entrega inclui gerador DIMACS, automacao experimental, reconstrucao da solucao, visualizacao e fontes do relatorio.

2. Variaveis proposicionais

Considere uma grade com R linhas e C colunas de celulas. Existem $(R+1)C$ arestas horizontais e $R(C+1)$ arestas verticais. Para cada aresta e , definimos uma variavel booleana x_e , verdadeira se e somente se a aresta pertence ao laco.

2.1 Mapeamento DIMACS

As horizontais $h_{i,j}$ sao indexadas para $0 \leq i \leq R$ e $0 \leq j < C$. As verticais $v_{i,j}$ usam $0 \leq i < R$ e $0 \leq j \leq C$:

$$idx(h_{i,j}) = 1 + iC + j$$

$$idx(v_{i,j}) = 1 + (R+1)C + i(C+1) + j$$

Logo, o total de variaveis e:

$$N = (R+1)C + R(C+1) = 2RC + R + C.$$

2.2 Exemplo 1x1

Uma grade 1x1 possui quatro arestas: $h_{0,0}=1$, $h_{1,0}=2$, $v_{0,0}=3$ e $v_{0,1}=4$. Uma dica 4 produz as clausulas unitarias (1), (2), (3) e (4). As clausulas de grau dos quatro cantos garantem que as duas arestas incidentes a cada canto tenham o mesmo valor.

2.3 Formato de entrada

```
3 3
3 . .
. 2 .
. . 3
```

A primeira linha informa R e C . Cada linha seguinte deve conter exatamente C valores; ponto representa uma celula sem dica. Valores fora de 0 a 4 sao rejeitados antes da geracao, evitando que uma entrada invalida seja silenciosamente interpretada como celula vazia.

3. Familias de clausulas e contagem

3.1 Cardinalidade das celulas

Para as quatro arestas $E=\{e_1, \dots, e_4\}$ de uma celula, codificamos exatamente k . Para $k=0$ ou $k=4$ sao usadas quatro clausulas unitarias. Para $k=1$, uma clausula exige ao menos uma aresta e seis clausulas binarias proíbem pares. Para $k=2$, quatro clausulas positivas sobre triplas exigem ao menos duas e quatro triplas negativas permitem no maximo duas. O caso $k=3$ e dual ao caso $k=1$.

Dica	Clausulas	Restricao
0	4	todas as arestas falsas
1	7	ao menos uma e no maximo uma
2	8	ao menos duas e no maximo duas
3	7	ao menos tres e no maximo tres
4	4	todas as arestas verdadeiras

3.2 Grau dos vertices

Em cada vertice sao permitidos somente graus 0 e 2. Cantos, com duas arestas incidentes, usam uma equivalencia de duas clausulas. Vertices de borda, com tres arestas, usam quatro clausulas para proibir graus 1 e 3. Vertices internos usam nove clausulas para proibir graus 1, 3 e 4. Essas condicoes sao necessarias e suficientes localmente: toda aresta ativa continua por outra aresta, sem ramificacao.

3.3 Formula geral

Se n_k e a quantidade de celulas com dica k , para $R, C \geq 1$:

$$M_{\text{grau}} = 8 + 4(2R+2C-4) + 9(R-1)(C-1)$$

$$M_{\text{base}} = M_{\text{grau}} + 4n_0 + 7n_1 + 8n_2 + 7n_3 + 4n_4 + 1$$

O termo final e a clausula que exige ao menos uma aresta ativa. Depois do refinamento, $M_{\text{final}}=M_{\text{base}}+q$, onde q e o numero de cortes de subtour encontrados durante a execucao.

4. Implementacao

4.1 Gerador e verificacao DIMACS

O arquivo `gerador.py` constroi as clausulas e escreve o CNF na saida padrao. Antes da emissao, a implementacao verifica dimensoes, dominio das dicas, intervalo dos literais, terminadores e correspondencia entre o cabeçalho e as clausulas.

```
def generate_dimacs(self):
    self.validate()
    header = f"p cnf {self.num_vars} {len(self.clauses)}"
    lines = [comment, header, *serialized_clauses]
    output = "\n".join(lines)
    validate_dimacs(output)
    return output
```

4.2 Refinamento CEGAR

As restricoes de grau garantem que cada componente ativo seja um ciclo, mas nao garantem um unico ciclo. Depois de cada resposta SAT, `refinamento.py` encontra componentes por busca em profundidade. Se houver mais de um, seleciona o menor componente S e adiciona:

$$\forall e \in S \neg x_e$$

A clausula afirma que ao menos uma aresta daquele subtour deve mudar. Ela e segura porque um ciclo isolado ja consome grau 2 em todos os seus vertices; qualquer laco global que passe por esses vertices precisa remover ao menos uma de suas arestas. O processo termina porque existe um numero finito de atribuicoes e cada corte elimina o modelo corrente.

```
componentes = detectar_componentes_conexos(arestas)
if len(componentes) == 1:
    return solucao
subtour = menor_componente(componentes)
clausulas.append(
    gerar_clausula_bloqueio(subtour, arestas, var_por_aresta)
)
```

4.3 Organizacao e testes

O runner usa arquivos temporarios fora do repositorio, aceita limite de tempo e iteracoes e exporta CSV. A suite automatizada cobre parser, cardinalidade 1x1, cabeçalho DIMACS, componentes artificiais e integracao real com os solvers empacotados.

5. Experimentacao

Foram executados CaDiCaL e Kissat em seis instancias. O tempo apresentado e o acumulado das chamadas do solver durante o refinamento, medido com relógio monotonicamente crescente. Base indica clausulas anteriores aos cortes; q indica cortes adicionados. Os binarios e o CSV bruto acompanham o repositório.

Instancia	Solver	N	Base	q	Iter.	Resultado	ms
instancia1_3x3_sat	CaDiCaL	24	99	0	1	SAT	1,511
instancia1_3x3_sat	Kissat	24	99	0	1	SAT	0,865
instancia2_4x4_unsat	CaDiCaL	40	186	0	1	UNSAT	1,379
instancia2_4x4_unsat	Kissat	40	186	0	1	UNSAT	0,579
instancia3_5x5_sat	CaDiCaL	60	305	1	2	SAT	3,113
instancia3_5x5_sat	Kissat	60	305	2	3	SAT	3,625
instancia4_6x6_unsat	CaDiCaL	84	537	0	1	UNSAT	1,350
instancia4_6x6_unsat	Kissat	84	537	0	1	UNSAT	1,133
instancia5_3x3_unsat	CaDiCaL	24	85	0	1	UNSAT	1,112
instancia5_3x3_unsat	Kissat	24	85	0	1	UNSAT	0,545
problema11_base	CaDiCaL	60	266	0	1	UNSAT	1,953
problema11_base	Kissat	60	266	0	1	UNSAT	1,042

Os tempos sao uma medicao do ambiente de desenvolvimento e nao um estudo estatistico. Em execucoes tao curtas, escalonamento do sistema e inicializacao do processo podem dominar a diferenca entre solvers.

6. Solucoes reconstruidas

A instancia 3x3 produz diretamente um unico laco. A instancia 5x5 e mais informativa: o primeiro modelo satisfaz todas as restricoes locais, mas contem ciclos desconectados. No CaDiCaL foi necessario um corte; no Kissat, dois. A figura mostra a entrada, um modelo intermediario invalido e a solucao conectada retornada pelo Kissat.

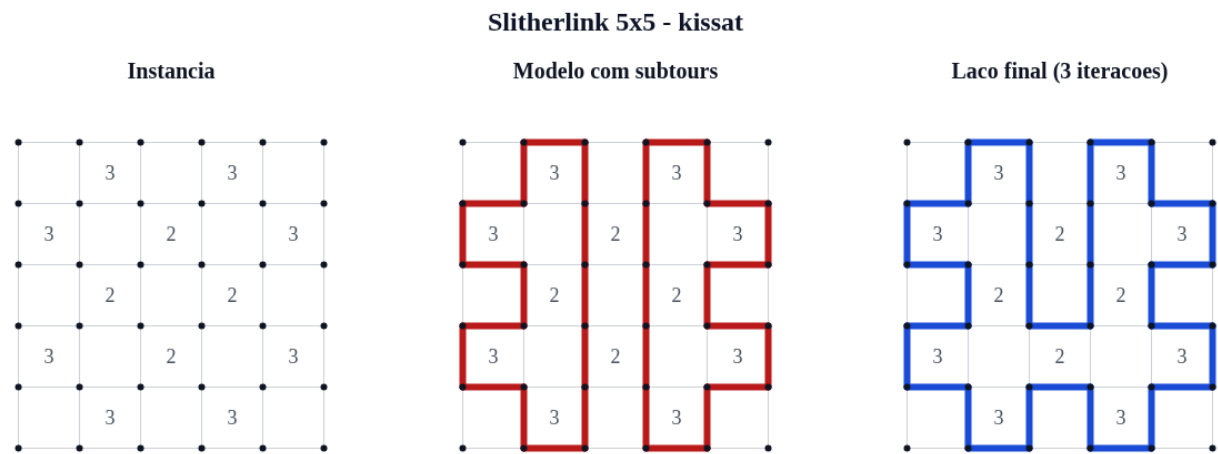


Figura 1 - Refinamento da instancia 5x5: entrada, contraexemplo e laco final.

```
3x3 SAT
+---+ + +
| 3 |
+ + + +
| | 2
+ +---+---+
| 3 |
+---+---+---+

5x5 SAT
```

```
+ +---+ +---+ +
| 3 | | 3 |
+---+ +---+ +---+
| 3 2 3 |
+---+---+---+
2 | 2 |
+---+---+---+
| 3 2 3 |
+---+ +---+ +---+
| 3 | | 3 |
+ +---+ +---+ +
```

A reconstrucao tambem funciona como verificacao independente do modelo: as variaveis positivas sao convertidas novamente em coordenadas horizontais e verticais, e a busca de componentes confirma que o resultado possui exatamente um laco.

7. Analise critica e conclusao

As instancias foram resolvidas em poucos milissegundos, indicando que a codificacao local e compacta para as grades estudadas. O resultado mais relevante nao e a diferenca absoluta de tempo, mas o fato de solvers distintos escolherem modelos iniciais diferentes: na instancia 5x5, o CaDiCaL precisou de um corte e o Kissat de dois. Isso evidencia que o custo do refinamento depende da busca interna, nao apenas de R e C.

7.1 Formalizacao alternativa

A conectividade poderia ser codificada estaticamente com variaveis auxiliares de alcance, fluxo ou ordem, escolhendo uma raiz e exigindo que toda aresta ativa seja alcancavel. Essa alternativa produz um unico CNF, mas adiciona muitas variaveis e clausulas. Tambem seria possivel aplicar Tseitin a subformulas de cardinalidade ou usar contadores sequenciais. Para quatro arestas por celula, a enumeracao direta e pequena e transparente; para restricoes maiores, contadores seriam mais economicos.

O refinamento adotado mantem a formula base compacta e adiciona somente cortes observados. Sua desvantagem e executar o solver varias vezes e tornar M_{final} dependente dos modelos encontrados. Para as instancias avaliadas, foram necessarios no maximo dois cortes, favorecendo a abordagem incremental.

7.2 Contribuicoes

Integrante	Contribuicao	Registro principal
Arthur Oliveira	Gerador, codificacao de celulas e graus, estrutura inicial	67ea8fa, dda3b3e e commits subsequentes da main
Joao Caique	Componentes, bloqueio de subtours e prototipo CEGAR	36321c8, 03123ea, 04b8377
Gustavo A. dos Santos	Requisitos, integracao final, experimentos, visualizacao e relatorio	e650e56 e revisao final

Referencias

[1] AVIGAD, J. *Mathematical Logic and Computation*. Cambridge University Press, 2022.

[2] BIERE, A. et al. *Handbook of Satisfiability*. 2. ed. IOS Press, 2021.

[3] YATO, T. *On the NP-completeness of Slitherlink*. IPSJ SIG Notes, 2003.